

CP020003

ARTIFICIAL INTELLIGENCE



Khon Kaen University · 2026

Week 03

Supervised Learning

(From Feature Engineering to Deep Learning
— an end-to-end journey)

1



Instructor: Teerapong Panboonyuen (P'Kao)

https://github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1

What we'll cover, end to end

1

Recap

Feature engineering + a zero-shot AI feature from Hugging Face

2

Data Prep

Turning strings into vectors, and a proper stratified split

3

Classical ML

Decision Tree, Random Forest, XGBoost, Logistic Regression, SVM

4

Deep Learning

Convolution & CNNs — letting the model learn features itself

5

Evaluation

Confusion matrix, precision/recall/F1, ROC-AUC, error analysis

6

Real World

When a false positive matters more than a false negative — and vice versa

PART ONE

Recap from Week 2

Where we left off with the Spotify dataset, and one demo we didn't finish.

1

The Spotify audio-features dataset

One row = one track. Every column is an audio signal already extracted for us — this is the "raw material" we engineer features from.

track_name	danceability	energy	loudness	tempo	genre
Comedy	0.676	0.461	-6.75	87.9	acoustic
Ghost - Acoustic	0.420	0.166	-17.24	77.5	acoustic
Can't Help Falling In Love	0.266	0.060	-18.52	181.7	acoustic
Hold On	0.618	0.443	-9.68	119.9	acoustic

~20 numeric & categorical columns

- danceability, energy, valence — 0 to 1 scores
- loudness — decibels (negative values)
- tempo — beats per minute
- duration_ms, explicit, mode, key
- track_genre — our usual target label

Source: kaopanboonyuen/CP020003 · spotify-kku-dataset.csv

Five quick, high-value features

duration_minutes

```
duration_ms / 60000
```

Raw milliseconds are hard to reason about — minutes are human-friendly.

energy_dance_ratio

```
energy / danceability
```

Captures whether a track is more 'driving' or more 'groovy'.

loudness_norm

```
min-max scale loudness to [0,1]
```

Puts loudness on the same footing as the other 0–1 features.

is_explicit_flag

```
cast explicit (True/False) → 1/0
```

Booleans must become numbers before any model can use them.

tempo_bucket

```
bin tempo into slow / medium / fast
```

Turns a continuous signal into an interpretable category.

A new feature powered by a pretrained AI model

Last week's Section 9 introduced **zero-shot classification** with Hugging Face — a model can label text with categories it was *never trained on*.

```
from transformers import pipeline

# Small, CPU-friendly zero-shot model (~250MB)
zero_shot = pipeline(
    "zero-shot-classification",
    model="typeform/distilbert-base-uncased-mnli",
    device=-1, # -1 = force CPU
)

# Labels the model was never explicitly trained on
candidate_moods = [
    "happy", "sad", "romantic",
    "energetic", "chill", "aggressive"
]

result = zero_shot(track_name, candidate_moods)
track_mood = result["labels"][0] # new feature!
```

Why this matters

- No manual keyword rules — the model reasons about meaning
- Runs fast on a free Colab CPU, ~250MB download
- Output track_mood becomes a brand-new categorical feature
- A preview of Part 4: letting AI build features for us

PART TWO

Preparing Data for Models

Machine learning models only understand numbers. Let's get there properly.



Why we encode, and One-Hot Encoding

Models do matrix math — a column of text like "acoustic" or "pop" cannot be multiplied by a weight. We must turn categories into numbers first.

One-Hot Encoding — one binary column per category

genre	is_acoustic	is_pop	is_rock
acoustic	1	0	0
pop	0	1	0
rock	0	0	1

Use One-Hot when...

- Categories have no natural order (genre, country, weekday)
- The number of categories is small-to-moderate
- You want to avoid implying a false ranking, e.g. rock > pop
- Trade-off: many categories → many new columns ("curse of dimensionality")

What is One-Hot Encoding?



One-Hot Encoding

is technique
that converts



Categorical Values

into



Binary Vectors

Benefits:



No Ordinal
Relationship



Improves
Model Accuracy



Works with Most
ML Algorithms



Simple and
Widely Supported

Consider a column called "Color" with three categories:

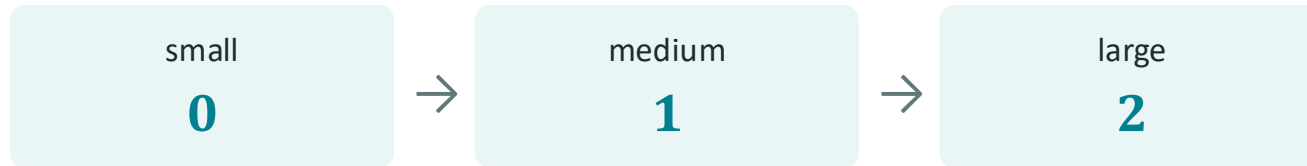
- Red
- Blue
- Green

After one-hot encoding, it becomes:

Color_Red	Color_Blue	Color_Green
1	0	0
0	1	0
0	0	1

Label / Ordinal Encoding — and which to pick

When categories DO have a natural order, map them directly to integers instead — no need for extra columns.



$s = 0, m = 1, l = 2$ — the gap between codes should mean something (e.g. "large" really is more than "medium")

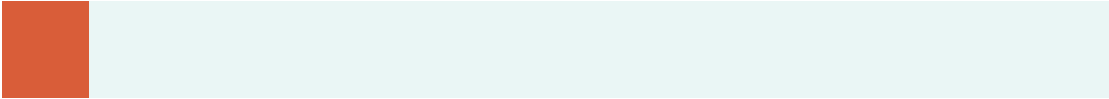
Quick decision guide

Question	One-Hot Encoding	Label / Ordinal Encoding
Is there a natural order?	No	Yes
Tree-based model (RF, XGBoost)?	Works, but not required	Works very well
Linear / distance model (LogReg, SVM, NN)?	Safer default	Risky unless order is real
Many categories (50+)?	Column explosion	Stays compact

Stratified Train / Test Split

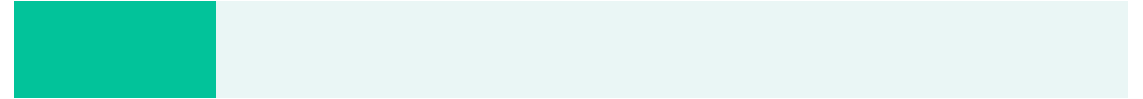
A random split can accidentally leave a rare class almost entirely out of the training set. Stratified splitting keeps the class ratio consistent in both sets.

Plain random split (can be unlucky)



train: only 4% minority class

Stratified split (matches original ratio)



train: same ~18% minority class as full data

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y    # <- key argument
)
```

Rule of thumb: whenever the target column is categorical (our genre / mood / type labels), stratify by y.

Before You Touch a Model: The Checklist



Missing values

Impute or drop — models can't accept NaN.



Encoding done

Every text / boolean column is now numeric.



Feature scaling

Scale for SVM, Logistic Regression, Neural Nets (trees don't need it).



Class balance check

Know your class ratio — it drives your split AND your metric choice.



No data leakage

Fit scalers/encoders on train only, then transform test.



Train/test split

Stratified, fixed random_state for reproducibility.

PART THREE

Classical Machine Learning Models

Six workhorse algorithms — how each one thinks about the data.

3

Six models, six ways of thinking

1

Decision Tree

Asks yes/no questions about features, splitting data until each leaf is (mostly) one class.

2

Random Forest

Trains many decision trees on random subsets, then lets them vote — reduces overfitting.

3

XGBoost

Builds trees one at a time; each new tree corrects the mistakes of the ones before it.

4

Logistic Regression

Fits a weighted sum of features, squashed into a 0–1 probability with a sigmoid curve.

5

SVM

Finds the boundary line that keeps the widest possible margin between classes.

6

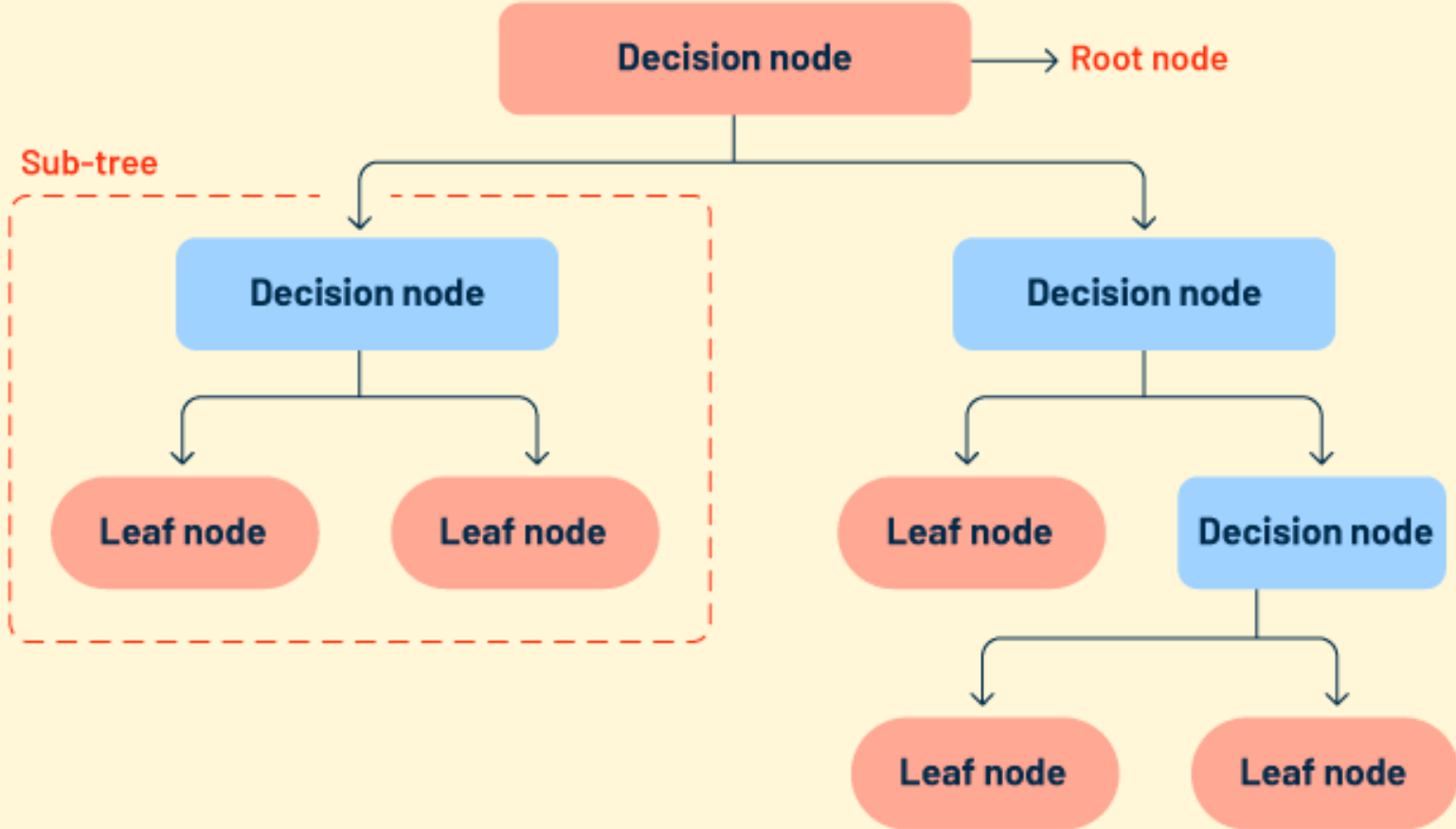
Neural Network (MLP)

Stacks layers of weighted connections and non-linear activations to learn patterns.

Today's lab: train all six on the Spotify features, then compare their scores side by side.



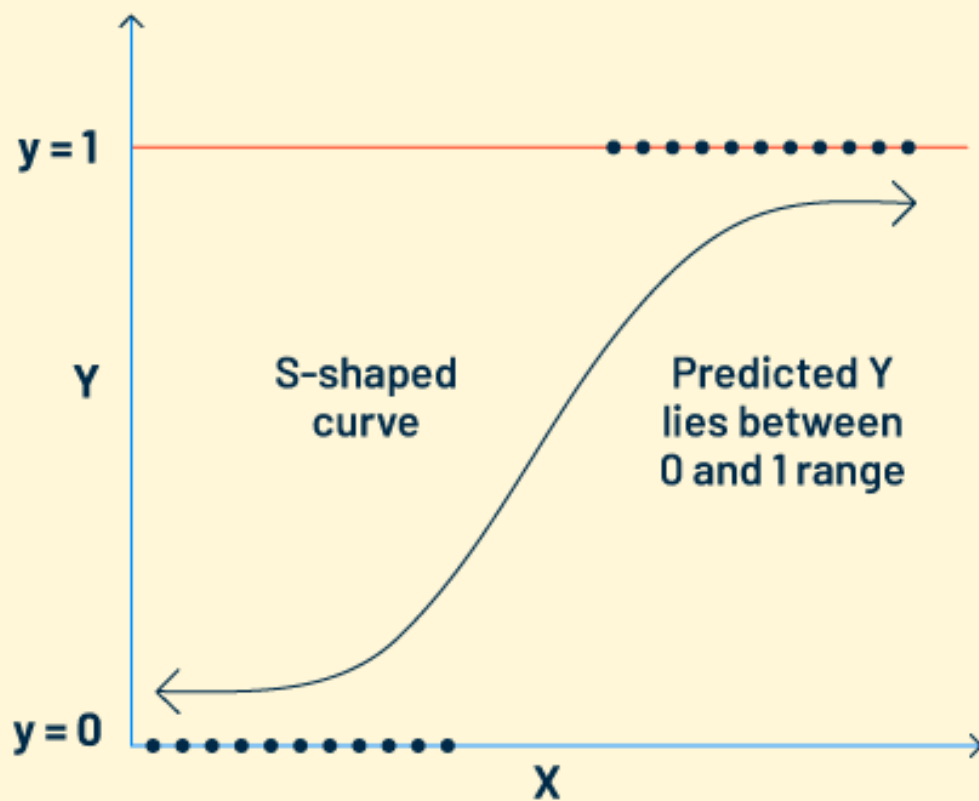
Decision trees





G2.com

Logistic Regression



$$P(Y = 1) = \frac{1}{1 + e^{-z}}$$

Predicting Whether a Song is a Hit Using Liveness

Target Variable (Y)

The Spotify dataset does not contain a column named **Hit**. Therefore, we create a binary target variable from the **popularity** score.

$$Y = \begin{cases} 1, & \text{if Popularity} \geq 70 \quad (\text{Hit}) \\ 0, & \text{if Popularity} < 70 \quad (\text{Not Hit}) \end{cases}$$

Feature (X)

We choose **liveness** as the predictor.

What is Liveness?

Liveness measures the likelihood that a track was performed live.

- Values range from **0 to 1**.
- A higher value indicates a greater probability that the recording contains a live audience.
- According to Spotify, a value above **0.8** generally suggests the track is likely a live performance.

For example:

Song	Liveness
Comedy	0.358
Ghost	0.101
Hold On	0.083
Lucky	0.151

Logistic Regression Model

The linear predictor is

$$z = \beta_0 + \beta_1(\text{Liveness})$$

The sigmoid function converts this into the probability that a song is a hit:

$$P(\text{Hit} = 1) = \frac{1}{1 + e^{-z}}$$

Substituting z :

$$P(\text{Hit} = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{Liveness})}}$$

Decision Rule

The predicted probability is converted into a class label using a threshold of 0.5.

If

$$P(\text{Hit} = 1) \geq 0.5,$$

then

Predict: Hit (1)

Otherwise,

Predict: Not Hit (0)

Interpretation

The coefficient β_1 indicates how **liveness** affects the probability that a song becomes a hit.

- If $\beta_1 > 0$, higher liveness increases the probability of being a hit.
 - If $\beta_1 < 0$, higher liveness decreases the probability of being a hit.
 - If $\beta_1 \approx 0$, liveness has little or no effect on predicting whether a song is a hit.
-

Example

Suppose the fitted model is

$$z = -1.8 + 3.2(\text{Liveness}).$$

For a song with

$$\text{Liveness} = 0.20,$$

the linear predictor is

$$z = -1.8 + 3.2(0.20) = -1.16.$$

The predicted probability is

$$P(\text{Hit} = 1) = \frac{1}{1 + e^{1.16}} \approx 0.24.$$

Since

$$0.24 < 0.5,$$

the model predicts that the song is **Not Hit (0)**.

Summary

- **Feature (X):** Liveness
- **Target (Y):** Hit (1) or Not Hit (0), created from the popularity score
- **Model:**

$$P(\text{Hit} = 1) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \text{Liveness})}}$$

- **Decision Boundary:** The model predicts **Hit** when the estimated probability is at least **0.5**.

This is a complete one-feature Logistic Regression example using **liveness** from the Spotify dataset.

The screenshot shows a web browser window with the URL `scikit-learn.org/stable/supervised_learning.html`. The page features a top navigation bar with links for 'Install', 'User Guide', 'API', 'Examples', 'Community', and 'More'. A search bar and a version selector (1.9.0 (stable)) are also present. The left sidebar contains a 'Section Navigation' menu with a 'Collapse Sidebar' button. The main content area is titled '1. Supervised learning' and lists various models and techniques. A right sidebar contains a promotional message for Probabl.

Section Navigation

- 1. Supervised learning
 - 1.1. Linear Models
 - 1.2. Linear and Quadratic Discriminant Analysis
 - 1.3. Kernel ridge regression
 - 1.4. Support Vector Machines
 - 1.5. Stochastic Gradient Descent
 - 1.6. Nearest Neighbors
 - 1.7. Gaussian Processes
 - 1.8. Cross decomposition
 - 1.9. Naive Bayes
 - 1.10. Decision Trees
 - 1.11. Ensembles: Gradient boosting, random forests, bagging, voting, stacking
 - 1.12. Multiclass and multioutput algorithms
 - 1.13. Feature selection
 - 1.14. Semi-supervised learning
 - 1.15. Isotonic regression
 - 1.16. Probability calibration
 - 1.17. Neural network models (supervised)
- 2. Unsupervised learning

1. Supervised learning

1.1. Linear Models

- [1.1.1. Ordinary Least Squares](#)
- [1.1.2. Ridge regression and classification](#)
- [1.1.3. Lasso](#)
- [1.1.4. Multi-task Lasso](#)
- [1.1.5. Elastic-Net](#)
- [1.1.6. Multi-task Elastic-Net](#)
- [1.1.7. Least Angle Regression](#)
- [1.1.8. LARS Lasso](#)
- [1.1.9. Orthogonal Matching Pursuit \(OMP\)](#)
- [1.1.10. Bayesian Regression](#)
- [1.1.11. Logistic regression](#)
- [1.1.12. Generalized Linear Models](#)
- [1.1.13. Stochastic Gradient Descent - SGD](#)
- [1.1.14. Robustness regression: outliers and modeling errors](#)
- [1.1.15. Quantile Regression](#)
- [1.1.16. Polynomial regression: extending linear models with basis functions](#)

1.2. Linear and Quadratic Discriminant Analysis

- [1.2.1. Dimensionality reduction using Linear Discriminant Analysis](#)
- [1.2.2. Mathematical formulation of the LDA and QDA classifiers](#)

scikit-learn is [financially supported](#) by Probabl and other institutions.

:probabl.

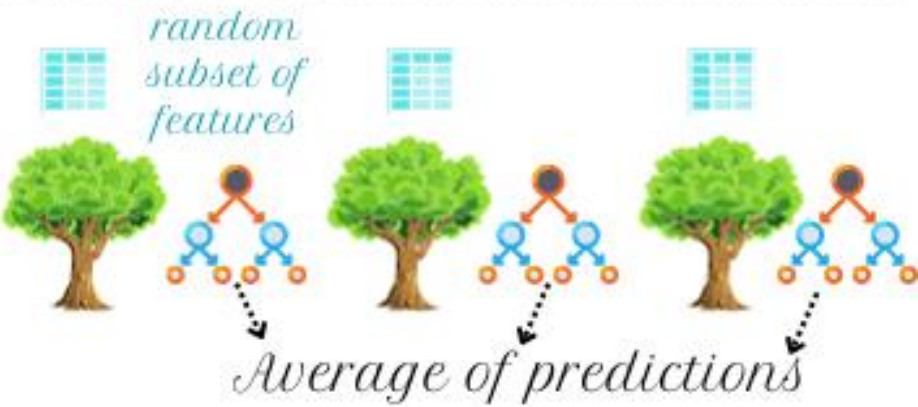
[Enterprise-grade solutions and services](#)

Decision Tree



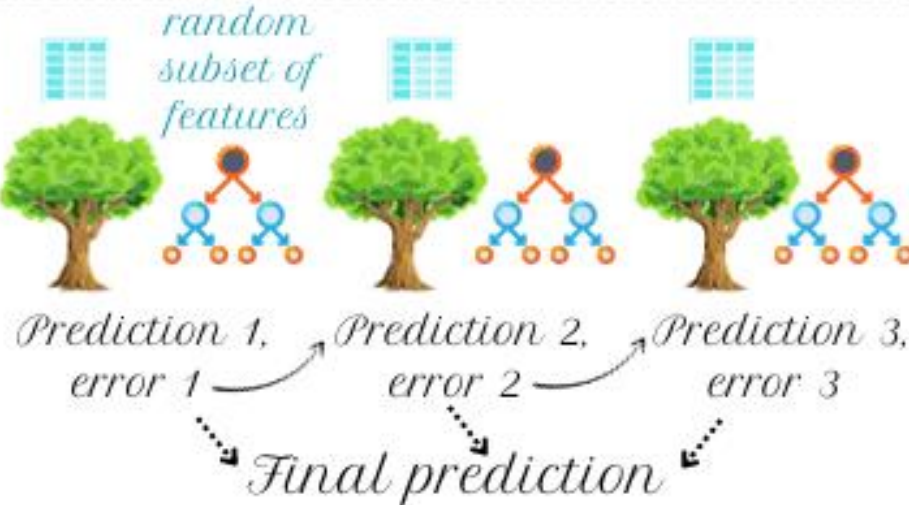
Random Forest

Parallel training



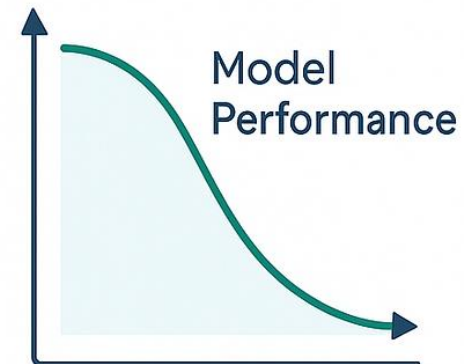
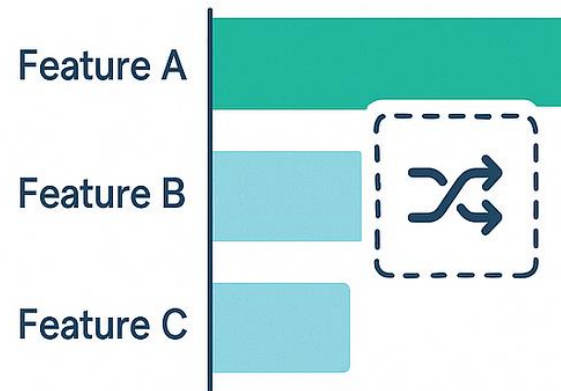
XGBoost

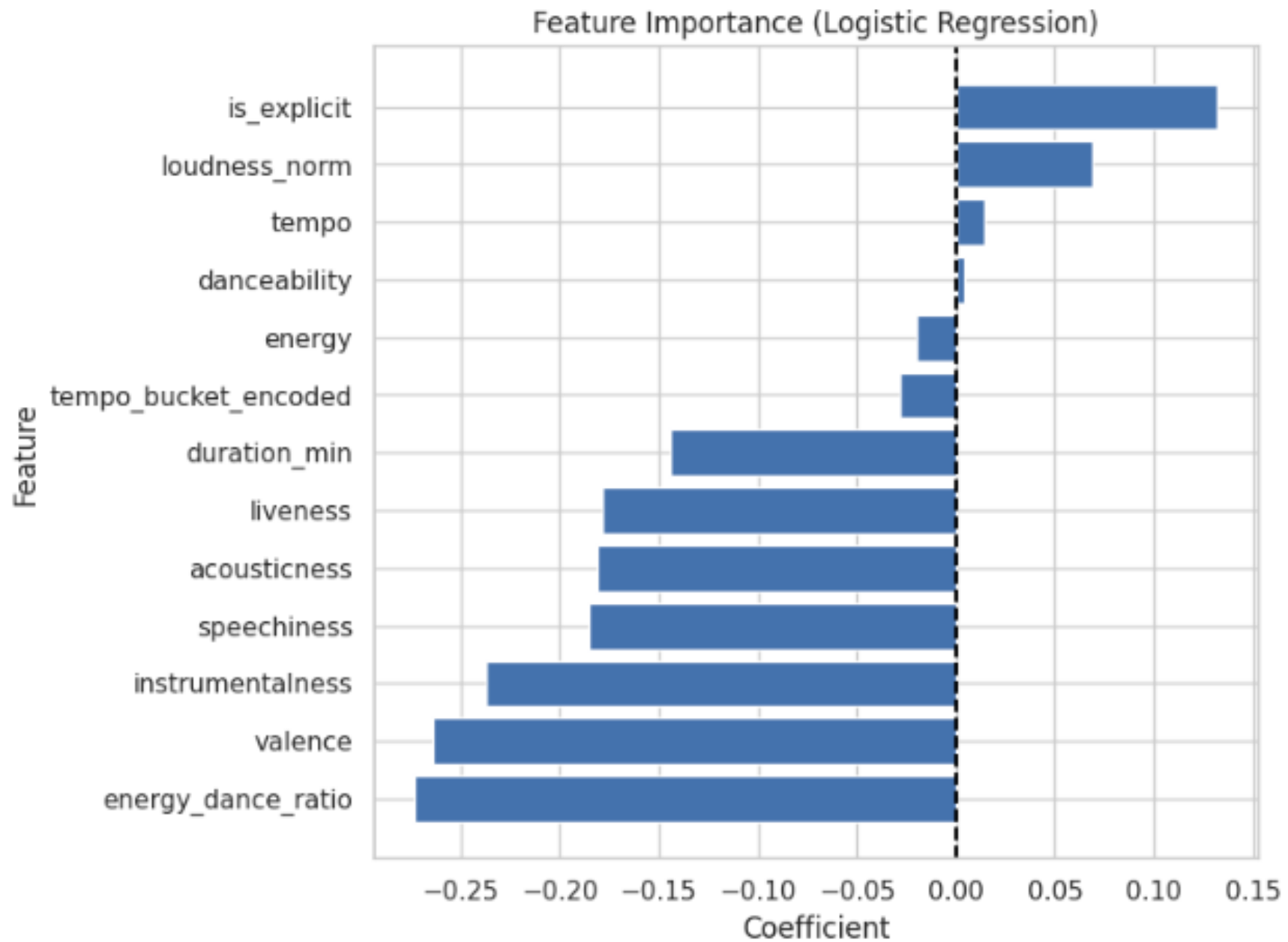
Sequential training



Interpreting ML Models

Feature Importance





PART FOUR

Deep Learning: Let the Machine Find Features

From handcrafted features to features the model learns by itself.





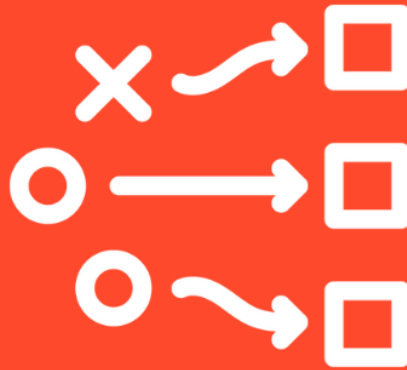
G2.com

Algorithm



A set of programming instruction to execute the task at hand.

Machine learning



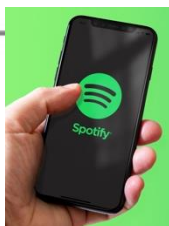
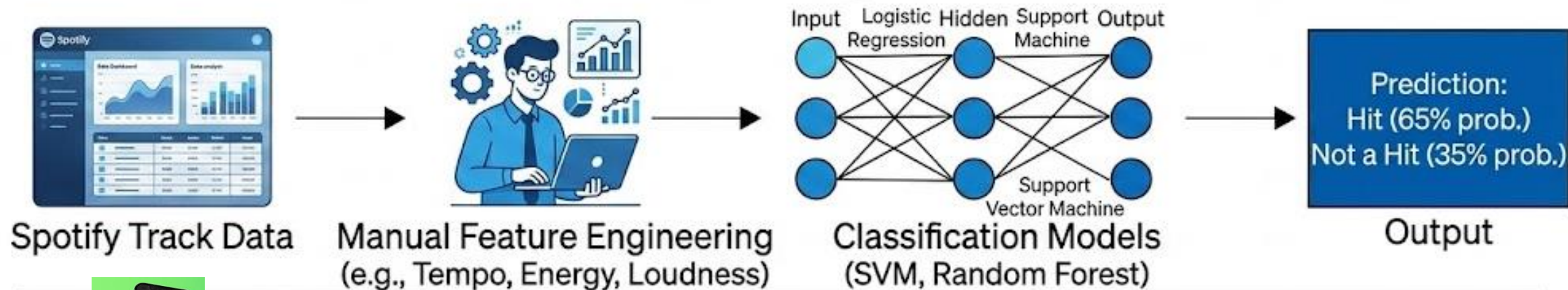
An AI technique to use data and algorithms to simulate human intelligence.

Deep learning

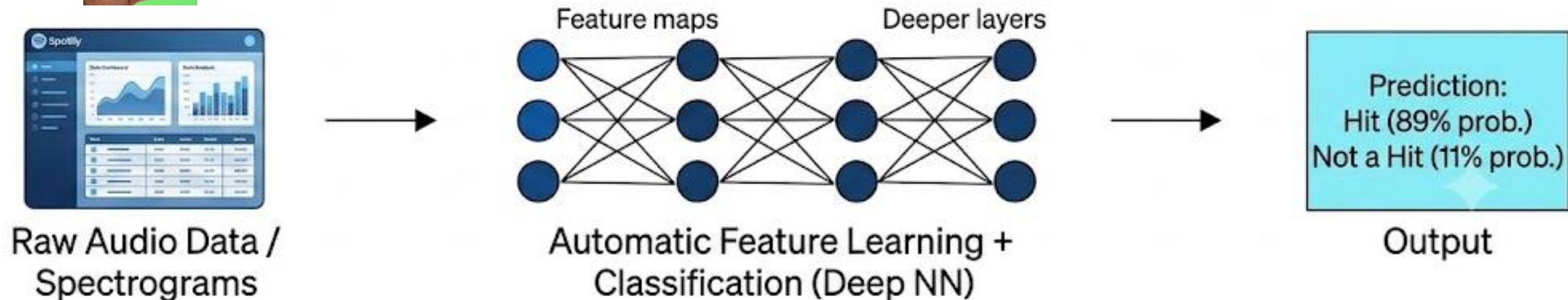


Artificial neural network that has multiple layers process high level features of data.

Machine Learning (Traditional)



Deep Learning



Manual feature engineering vs automatic feature learning

Classical ML — you craft the features

- You decide: energy_dance_ratio, loudness_norm, tempo_bucket...
- Requires domain knowledge and trial and error
- Features are easy to inspect and explain
- Ceiling on performance = ceiling on your creativity

VS

Deep Learning — the model crafts the features

- Convolution kernels learn which local patterns matter
- Needs more data and compute, less domain guesswork
- Learned features are powerful but harder to inspect
- Ceiling on performance = ceiling on data + architecture

What is convolution? A CNN, layer by layer

Convolution in one sentence

A small window (the "kernel") slides across the input, multiplying and summing values at each position — it's a pattern detector that looks for the same local shape everywhere in the data.

Why it beats handcrafting

Instead of us guessing "maybe loudness matters," the kernel weights are learned from data during training — the network discovers which patterns actually predict the label.



Classical ML models use the features you engineered (Slide 5). A CNN can take a rawer signal and learn its own — we'll compare both approaches head-to-head today.

PART FIVE

Evaluating Your Model

Training a model is easy. Knowing whether it's actually good is the real skill.

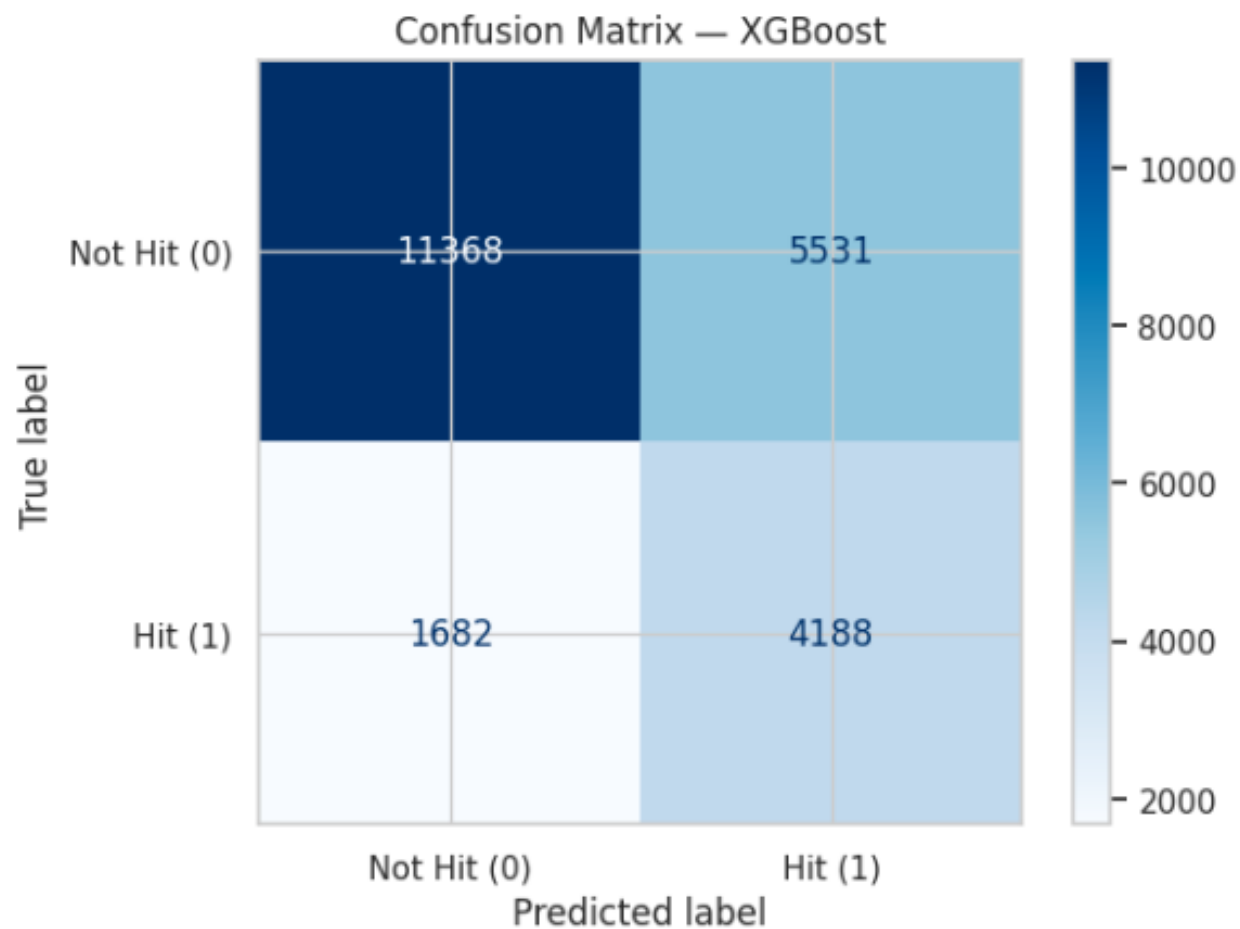


The Confusion Matrix: TP, FP, FN, TN

	Predicted: Positive	Predicted: Negative
Actual: Positive	TP True Positive correctly predicted YES	FN False Negative missed a real YES
Actual: Negative	FP False Positive wrongly predicted YES	TN True Negative correctly predicted NO

Every prediction lands in exactly one of these four boxes — every metric on the next slide is built from these four counts.

Best model by F1: XGBoost



$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad \text{— overall, how often are we correct?}$$

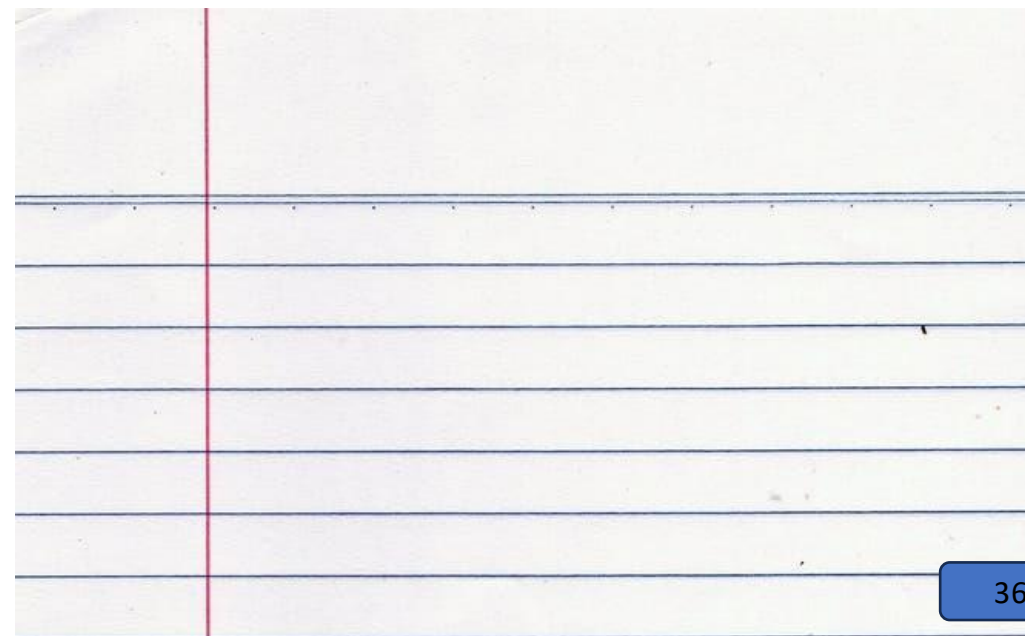
$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{— of everything we called "Hit", how many really were?}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{— of all the actual Hits, how many did we catch?}$$

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad \text{— balances Precision and Recall into one number}$$

AUC (ROC) = how well the model ranks positives above negatives, across every possible threshold

	train_seconds	accuracy	f1
Random Forest	27.972	0.613	0.488
Decision Tree	2.182	0.550	0.443
XGBoost	1.531	0.683	0.537
Logistic Regression	0.544	0.570	0.429



Turning TP / FP / FN / TN into metrics

Accuracy

$$(TP + TN) / (TP + TN + FP + FN)$$

Overall, how often is the model right? Misleading on imbalanced data.

Precision

$$TP / (TP + FP)$$

Of everything predicted positive, how much was correct? Cares about false alarms.

Recall

$$TP / (TP + FN)$$

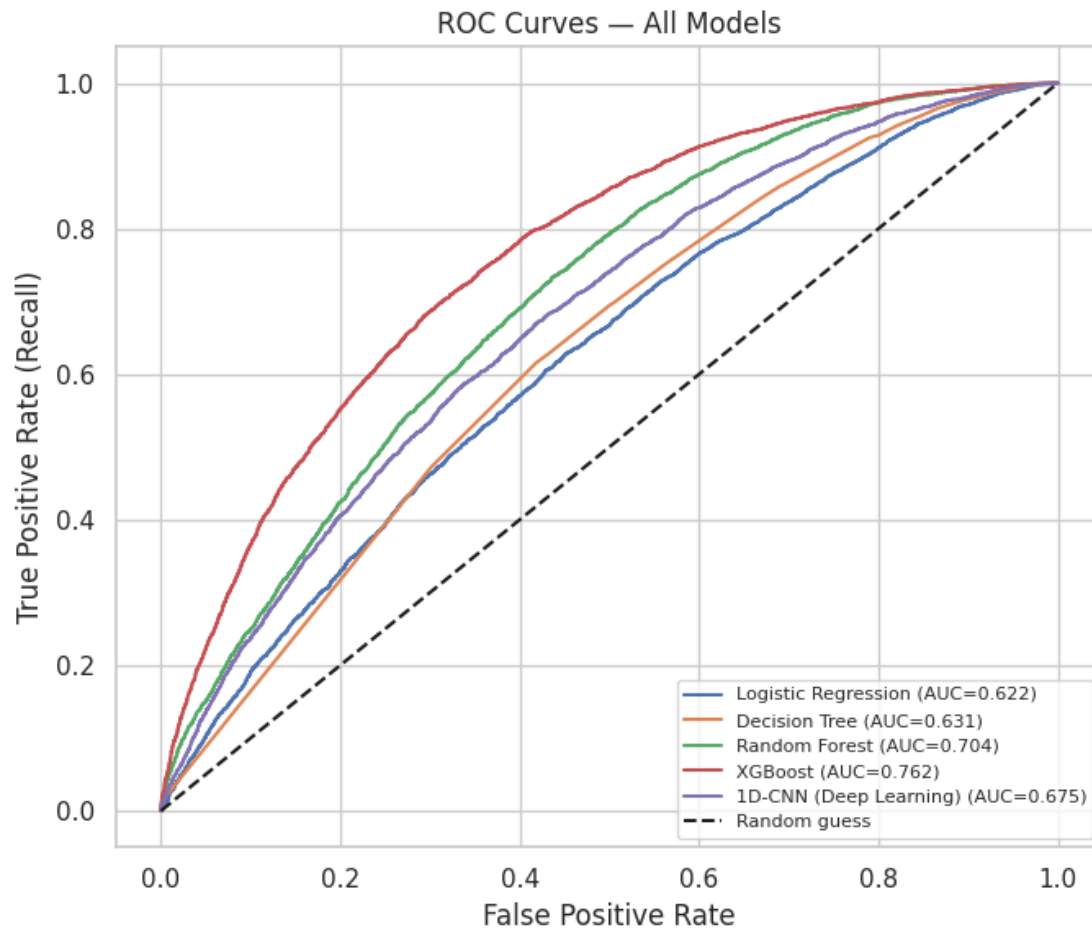
Of everything actually positive, how much did we catch? Cares about misses.

F1-score

$$2 \times (Precision \times Recall) / (Precision + Recall)$$

Harmonic mean — a single number balancing precision and recall.

ROC Curve & AUC



Reading the curve

- The curve plots True Positive Rate vs False Positive Rate at every possible decision threshold
- AUC = Area Under the Curve — a single number from 0.5 (random) to 1.0 (perfect)
- A curve that hugs the top-left corner means the model separates classes well
- Useful for comparing models independent of the exact threshold you'll deploy with

Case Study: Medical Diagnosis

A model screens patients for a serious disease. Which error should we fear more?

False Positive

Healthy patient flagged as sick → extra tests, stress, cost — but usually correctable once retested.

False Negative

Sick patient told they're healthy → disease goes untreated. Often far more dangerous.

In medicine we usually optimize for Recall (catch every real case), accepting more false alarms — because the cost of a missed diagnosis is much higher than the cost of an extra test.

Same idea, different businesses

Spam filter

Worry more about FP

A real email marked as spam (FP) may be missed forever. A spam email in the inbox (FN) is just annoying.

Fraud detection

Worry more about FN

Missing real fraud (FN) costs real money. A false alarm (FP) just means one extra manual review.

Churn prediction

Balance both

Too many FPs waste retention offers on happy customers; too many FNs mean losing customers we could have saved.

Error analysis and comparing every model

After metrics, always look at the mistakes themselves — patterns in errors point to what to fix next.

1

Confusion matrix heatmap

Which classes get confused with which?

2

ROC curves overlaid

Compare all 6 models on one chart

3

Feature importance

Which engineered features actually helped?

4

Misclassified samples

Read a few by hand — is there a pattern?

Final deliverable for today's lab

Model	Accuracy	Precision	Recall	F1	AUC
Decision Tree	?	?	?	?	?
Random Forest	?	?	?	?	?
XGBoost	?	?	?	?	?
Logistic Regression	?	?	?	?	?
SVM	?	?	?	?	?
CNN (auto features)	?	?	?	?	?

The end-to-end supervised learning pipeline



You now have the full loop — from a raw CSV to a defensible, evaluated model. Next: run the exact same loop on a new dataset, on your own.

NETFLIX

HOMEWORK

The Netflix Challenge

Apply everything from today to a brand-new dataset, independently.

6

Netflix catalogue — your turn, end to end

Dataset

netflix-kku-dataset.csv — show_id, type, title, director, cast, country, date_added, release_year, rating, duration, listed_in, description

Task

Choose a target column and predict it — for example: **type** (Movie vs TV Show) from the other columns. Define your own X (input features) and y (target) with a short justification.

Requirements

- Feature-engineer the raw columns (dates, cast/genre lists, duration text, etc.)
- Encode categorical columns — justify one-hot vs. label encoding for each
- Stratified train/test split
- Compare at least 5 models (mix of classical ML and/or deep learning)
- Report accuracy, precision, recall, F1, and ROC-AUC for every model
- Include a confusion matrix and 2–3 sentences of error analysis

★ Bonus points

- Use an ML model NOT taught in class (e.g. LightGBM, CatBoost, KNN)
- Or design your own deep learning architecture beyond the class CNN
- Runs cleanly on the free Colab tier, reasonable runtime

Let's Build End-to-End Models

Questions before we open Colab?

`github.com/kaopanboonyuen/CP020003_ArtificialIntelligence_2026s1`